Manual Técnico del Motor de Asignación Automática de Horarios de Clases

Documento 3 de la Entrega Final del Módulo 3. Arquitectura, Despliegue, Operación y Mantenimiento

Jesús David Beltrán Zárate

Juan Sebastián Albarracín Isaza

Daniel Felipe Delgadillo Sepúlveda

Martín Daniel Porto Gutiérrez

Programa de Ingeniería de Sistemas, Facultad de Ingeniería, Universidad El Bosque

Proyecto Núcleo 2 (15319), 2026-2, Grupo 5

Ing. Samuel Galindo Pinzón

5 de noviembre de 2026

Repositorio https://github.com/jesusBeltran79/ProgHorarioClase

Normas APA 7.ª edición

Introducción

Este manual explica cómo está construido el motor de horarios del Grupo 5 y cómo se instala, se opera y se mantiene. Está escrito para quien reciba el sistema después del equipo y tenga que levantarlo desde cero o cambiar una regla. Parte del diseño aprobado en el Módulo 2 en su versión construida. No repite la justificación del diseño. Esa justificación está en el Documento 1. Informe de Ingeniería (PN2_Grupo5_Doc1_Informe.pdf). Los diagramas UML y el diagrama entidad relación están en el Documento 2. Anexos Técnicos (PN2_Grupo5_Doc2_Anexos.pdf). La operación desde la interfaz web para el administrador está en el Documento 4. Manual de Usuario (PN2_Grupo5_Doc4_ManualUsuario.pdf).

Todos los nombres de clases, paquetes, rutas y tablas de este documento se tomaron del código del repositorio. Si un nombre de este manual no coincide con el código, manda el código y el manual debe corregirse.

Requisitos del Sistema

El sistema corre en tres contenedores. Uno sirve la interfaz web, otro ejecuta el backend y el tercero guarda los datos. La Tabla 1 fija las versiones de cada pieza. Son las mismas que declaran `backend/pom.xml` y `frontend/package.json`.

Tabla 1

Versiones de lenguajes, frameworks y bases de datos

Capa	Tecnología	Versión
Lenguaje del backend	Java (Eclipse Temurin)	21
Framework del backend	Spring Boot con los módulos web, jdbc, security, validation y actuator	3.3.5
Migraciones de la base	Flyway (flyway-core y flyway-database-postgresql)	La que gestiona Spring Boot 3.3.5
Tokens de sesión	JJWT (io.jsonwebtoken) api, impl y jackson	0.12.6
Base de datos	PostgreSQL	16
Base de pruebas y de respaldo	H2 en modo PostgreSQL	La que gestiona Spring Boot 3.3.5
Construcción del backend	Apache Maven	3.9 o superior (3.9.9 en la imagen de Docker)
Interfaz web	Angular con componentes standalone	18.2

Capa	Tecnología	Versión
Compilación de la interfaz	Node.js	20 LTS
Servidor de la interfaz	nginx	1.27
Contenedores	Docker Engine con Docker Compose v2. Imágenes base postgres:16-alpine, maven:3.9.9-eclipse-temurin-21, eclipse-temurin:21-jre, node:20-alpine y nginx:1.27-alpine	Compose v2
Integración continua	GitHub Actions (ubuntu-latest)	No aplica

Nota. Las versiones exactas de las librerías que gestiona Spring Boot y el equipo donde se tomaron las mediciones quedan registrados en docs/evidencias/entorno.txt.

Las pruebas y las mediciones del equipo corrieron en GitHub Actions sobre ubuntu-latest con 4 núcleos Intel Xeon Platinum 8370C a 2,80 GHz y 15 GB de memoria. La máquina virtual fue OpenJDK Temurin 21.0.13 y la construcción usó Apache Maven 3.10.0, como registra docs/evidencias/entorno.txt. En el arranque con Docker Compose el registro del backend muestra Java 21.0.13, PostgreSQL 16.15, Apache Tomcat 10.1.31 y nginx 1.27.5. Flyway validó y aplicó las dos migraciones sobre la base vacía (docs/evidencias/sistema/docker_compose.log).

Para instalar con contenedores basta un equipo con Docker Desktop o Docker Engine, 4 GB de memoria libre y unos 3 GB de disco para las imágenes. No hace falta instalar Java ni Node.js en el equipo porque las imágenes compilan todo por dentro. Para trabajar sin Docker sí se necesitan Java 21, Maven 3.9 y Node.js 20. El navegador del administrador puede ser cualquier versión reciente de Chrome, Edge o Firefox.

El sistema usa tres puertos. La interfaz web responde en el 4200. El backend responde en el 8080. PostgreSQL escucha en el 5432 con la base `horarios`, el usuario `horarios` y la clave `horarios`. Si alguno de esos puertos ya está ocupado en el equipo hay que liberarlo antes de arrancar.

Arquitectura del Motor

Capas y Paquetes

El backend es un monolito modular en cuatro capas. La presentación es la interfaz web en Angular. La aplicación expone la API REST y coordina cada caso de uso. El dominio contiene el validador, el motor y las restricciones. La persistencia lee los archivos CSV, guarda los inventarios y el historial de ejecuciones y exporta el horario. La Figura E1 del Documento 2. Anexos Técnicos (PN2_Grupo5_Doc2_Anexos.pdf) muestra los componentes y el nodo donde corre cada uno.

Cada capa vive en un paquete de `co.edu.unbosque.horarios`. La Tabla 2 lista los paquetes y las clases principales de cada uno. La regla más importante de la estructura es que los paquetes del núcleo no importan Spring. Así el motor es el mismo que se midió en el Módulo 2 y se puede probar sin levantar la aplicación ni la base de datos.

Tabla 2

Paquetes del backend y responsabilidad de cada uno

Paquete	Usa Spring	Clases principales y responsabilidad
dominio	No	Docente, Aula, Curso, Grupo, Sesion, FranjaHoraria, Asignacion, DiaSemana y CaracteristicaAula. Entidades inmutables del problema en memoria
configuracion	No	ConfiguracionParametros. Guarda los parámetros del semestre, genera las franjas válidas y calcula el umbral de RV-01
restricciones	No	Interfaz RestriccionDura y sus estrategias RD-01, RD-02, RD-04, RD-05, RD-06 y RD-10. RestriccionMismoDiaRD09 existe solo para la prueba de modificabilidad
validacion	No	ReglaValidacionEntrada, ReglaTopeMatriculadosRV01, ReglaCierreMinimoRV02, ValidadorEntrada y ReporteValidacion
motor	No	MotorAsignacion, EstadoHorario, OrdenExploracion, OrdenFranjas, ReporteConflicto y ResultadoAsignacion
datos	No	ConjuntoDatos y GeneradorDatosSinteticos para las mediciones con semilla fija
persistencia	Sí	CargadorInventariosCSV, LectorCSV, ConstructorDominio, ExportadorHorarioCSV y los repositorios JDBC RepositorioInventarios, RepositorioParametros, RepositorioEjecuciones, RepositorioUsuarios y RepositorioCatalogos

Paquete	Usa Spring	Clases principales y responsabilidad
aplicacion	Sí	ControladorInventarios, ControladorParametros, ControladorHorarios, sus servicios, ConfiguracionMotor, ManejadorErrores, ExcepcionNegocio y los DTO en dto.Dto. Subpaquete catalogos con ServicioCatalogos y ControladorCatalogos. Subpaquete cuentas con ServicioCuentas, ServicioPortal, ControladorUsuarios y ControladorPortal
seguridad	Sí	ConfiguracionSeguridad, FiltroJwt, ServicioJwt, ControladorAutenticacion e InicializadorDatos
app	No	Programas de consola para medir y demostrar. MedicionEscalabilidad, PruebaEstres, PruebaDeCambio, DemostracionConflicto y EjecutarConjuntoCSV

Nota. Los cuatro primeros paquetes y motor forman el núcleo. Los paquetes persistencia, aplicacion y seguridad lo envuelven para la versión web.

La interfaz web es un proyecto Angular 18.2 con componentes standalone. Tiene cuatro páginas, IngresoComponent, CargaComponent, ParametrosComponent y HorarioComponent, dentro de un MarcoComponent con el menú lateral. ApiService concentra las llamadas a la API. AutenticacionService guarda el token y un interceptor lo agrega en la cabecera de cada petición. Una guardia de rutas envía a la pantalla de ingreso a quien no tenga sesión.

La apariencia de toda la interfaz sale de un sistema de diseño propio con tonos verdes. Sus valores están como variables CSS al comienzo de `frontend/src/styles.css`, por ejemplo `--brand` para el color principal, `--choque` para los errores y `--aviso` para las advertencias, además de las fuentes, los espacios y los radios. Para cambiar el aspecto se editan esas variables y no cada pantalla. Cada pantalla trae un texto corto de ayuda que explica qué hacer en ella.

El módulo de usuarios agregó dos grupos de páginas. Las páginas de gestión del administrador están en `paginas/gestion` y son docentes, aulas, materias y usuarios. Las del docente están en `paginas/portal` y son mi horario, mi disponibilidad y mi cuenta. `GestionService` y `PortalService` llaman a la API de cada perfil. La grilla de disponibilidad es un componente reutilizable, `SelectorFranjasComponent`, que se pinta con clic o arrastrando. El arrastre de docentes entre listas y el de sesiones en la grilla del horario usan Angular CDK 18.2. La guardia `guardiaRol` deja entrar a cada ruta solo al perfil que corresponde y el menú cambia según el rol del token.

El acceso a la base se hace con `JdbcTemplate` de Spring y consultas SQL explícitas en los repositorios. No se usan entidades JPA. Con eso el esquema queda definido en un solo lugar, que son los scripts de Flyway, y el núcleo no carga anotaciones de persistencia.

Ensamblaje del Núcleo

La clase `ConfiguracionMotor` del paquete aplicacion es el único lugar donde se registran las reglas. Publica un bean `CatalogoRestricciones` con la lista de restricciones duras y un bean `ValidadorEntrada` con las reglas RV-01 y RV-02. `ServicioHorarios` recibe los dos por constructor y crea un `MotorAsignacion` nuevo en cada generación. El motor nunca crea reglas ni conoce sus clases concretas. Ese es el principio de inversión de dependencias (Martin, 2003) que el diseño del Módulo 2 había ubicado en ControladorHorarios. En la construcción pasó a una clase de configuración para que el controlador no tenga lógica.

```
@Bean
public CatalogoRestricciones catalogoRestricciones() {
    return new CatalogoRestricciones(List.of(
            new RestriccionDisponibilidadDocenteRD04(),
            new RestriccionIdoneidadAulaRD05(),
            new RestriccionCapacidadAulaRD06(),
            new RestriccionChoqueDocenteRD01(),
            new RestriccionChoqueAulaRD02(),
            new RestriccionDocenteUnicoGrupoRD10()));
}
```

Flujo de una Generación

Cuando el administrador pide generar un horario la petición llega a `POST /api/horarios/generar`. `ControladorHorarios` solo pasa el nombre del usuario autenticado a `ServicioHorarios.generar`. El servicio carga los parámetros vigentes con `RepositorioParametros` y genera la lista de franjas válidas. Luego reconstruye el problema en memoria con `ConstructorDominio` a partir de los inventarios guardados. Después corre `ValidadorEntrada` sobre los grupos y anota los hallazgos de RV-01 y RV-02, que no detienen la generación.

Con todo en memoria el servicio llama a `MotorAsignacion.generarHorario` con el orden de la sesión más restringida primero y los mapas de ocupación activos. Al terminar guarda

la ejecución con `RepositorioEjecuciones` en una sola transacción. Se guardan la fila de `ejecucion_horario`, las asignaciones, los conflictos con sus códigos y los hallazgos de validación. La respuesta es un objeto `Resultado` con el estado COMPLETO o PARCIAL. La Figura C1 del Documento 2. Anexos Técnicos (PN2_Grupo5_Doc2_Anexos.pdf) sigue este flujo como diagrama de secuencia.

Algoritmo de Asignación

El motor trata cada sesión como una variable cuyo dominio son las tuplas de docente, aula y franja, como en un problema de satisfacción de restricciones (Russell y Norvig, 2021). Trabaja en cuatro pasos. El primero cuenta para cada sesión cuántas tuplas pasan las restricciones locales, que son las que devuelven verdadero en `esLocal`. Hoy son RD-04, RD-05 y RD-06. El segundo ordena las sesiones de menor a mayor dominio para atender primero la más restringida. El tercero es el backtracking. Para cada sesión prueba tuplas y evalúa las reglas locales antes que las de choque porque son más baratas. Si ninguna tupla sirve deshace la última asignación y retrocede.

El cuarto paso solo ocurre si el árbol se agota o se supera el tiempo límite. El motor recupera la asignación parcial más profunda que alcanzó e intenta ubicar sobre ella las sesiones que faltan. Para cada sesión que sigue sin lugar evalúa todas las reglas sin detenerse en la primera falla y cuenta cuántas tuplas bloquea cada código. Con eso arma un `ReporteConflicto`. La Figura B1 del Documento 2. Anexos Técnicos (PN2_Grupo5_Doc2_Anexos.pdf) muestra los cuatro pasos como diagrama de actividades.

Hay dos detalles de la construcción que conviene conocer. El primero es que las franjas no se prueban en el orden de la grilla. `OrdenFranjas.intercalarDias` las ordena por hora y luego por día, de modo que después del lunes a las 07:00 se prueba el martes a las 07:00. Así las sesiones de un mismo grupo tienden a caer en días distintos sin agregar una regla. Ese orden no cambia qué horarios son válidos. Solo cambia cuál encuentra primero el motor. El segundo detalle es que el conteo inicial sigue recorriendo todas las tuplas de cada sesión. El precálculo

por franja que se planteó en el Módulo 2 no quedó implementado en esta versión y el conteo sigue siendo el paso más costoso del motor.

`EstadoHorario` guarda las asignaciones confirmadas y dos listas de conjuntos, una de docentes ocupados y otra de aulas ocupadas, con un conjunto por franja. Responder si un docente o un aula ya están ocupados cuesta una consulta a un conjunto. Al deshacer una asignación se quita de la lista y de los dos conjuntos. El tiempo límite del motor se lee de la propiedad `app.motor.limite-ms`, que por defecto vale 60.000 ms para cumplir RNF-01.

Reubicación y Alternativas

El ajuste manual no pasa por el backtracking. `ServicioHorarios.alternativas` reconstruye el horario guardado sin la sesión elegida y evalúa todas las tuplas contra las mismas restricciones del catálogo. Devuelve primero las tuplas válidas y después las que solo fallan por RD-01 o RD-02, con el campo `liberar` que indica qué choque habría que deshacer. `ServicioHorarios.reubicar` vuelve a evaluar la tupla que eligió el administrador. Si viola alguna regla responde 409 con el código `RESTRICCION_VIOLADA` y la lista de códigos RD. Si no viola ninguna actualiza la asignación y recalcula si el horario quedó completo.

La base de datos es la segunda barrera. La tabla `asignacion` tiene dos restricciones UNIQUE, una sobre ejecución, docente, día y hora de inicio y otra sobre ejecución, aula, día y hora de inicio. Un choque de docente o de aula que llegue por un camino distinto del motor lo rechaza PostgreSQL.

Seguridad

La seguridad es sin sesión de servidor. `ControladorAutenticacion` recibe usuario y contraseña en `POST /api/auth/login`, compara la contraseña con el hash bcrypt de la tabla `usuario` y entrega un token JWT firmado por `ServicioJwt`. El token dura 480 minutos por defecto. `FiltroJwt` lee la cabecera `Authorization: Bearer` en cada petición y deja autenticado al usuario con el rol que trae el token. `ConfiguracionSeguridad` exige el rol DOCENTE en `/api/mi/**` y el rol ADMIN en el resto de `/api`, salvo `GET`

`/api/parametros` que leen los dos perfiles. El token también lleva el docente al que está ligada la cuenta. El portal no confía en ese dato y busca el docente de la cuenta en la base con el nombre del usuario autenticado, así un docente nunca lee ni cambia datos de otro. Las únicas rutas públicas son el inicio de sesión y `/actuator/health`. Al arrancar, `InicializadorDatos` crea el usuario administrador si no existe y guarda su clave con bcrypt.

El horario exportado no lleva datos personales de estudiantes. El archivo de salida solo tiene curso, grupo, sesión, docente, aula, día y horas, como pide RNF-06.

API REST

La Tabla 3 resume las rutas. Todas devuelven JSON salvo la exportación, que devuelve `text/csv`. Los errores tienen siempre la forma `{codigo, mensaje, detalles}`, que arma `ManejadorErrores`.

Tabla 3

Rutas de la API REST del backend

Método y ruta	Respuesta	Requisito
POST /api/auth/login	Token con fecha de expiración	RNF-06
POST /api/inventarios (multipart con docentes, aulas, cursos y grupos)	ResumenCarga con cargados, rechazos y hallazgos	RF-01 a RF-05
GET /api/inventarios/resumen	ResumenCarga de la última carga	RF-05
GET /api/inventarios/catalogo	Docentes y aulas para la asignación manual	RF-07
GET /api/parametros	Parámetros y franjas válidas	RF-08
PUT /api/parametros	Parámetros recalculados	RF-08
POST /api/horarios/generar	Resultado COMPLETO o PARCIAL	RF-06, RF-07
GET /api/horarios	Lista de ejecuciones con usuario y fecha	RNF-06
GET /api/horarios/{id}	Resultado guardado	RF-06
GET /api/horarios/{id}/alternativas/{idSesion}	Lista de alternativas	RF-07
POST /api/horarios/{id}/reubicar	Resultado o 409 con códigos RD	RF-07, RNF-04
GET /api/horarios/{id}/exportar	Archivo CSV o 409 si el horario es PARCIAL	RF-09, RD-08
POST /api/horarios/{id}/publicar	Resultado publicado o 409 HORARIO_PARCIAL	RF-12

Método y ruta	Respuesta	Requisito
GET, POST, PUT y DELETE /api/admin/docentes, aulas, materias y grupos	El recurso, 204 al borrar, 409 DUPLICADO, 400 RD-03, RD-07, REFERENCIA o FORMATO y 404 NO_ENCONTRADO	RF-10
GET, POST, PUT y DELETE /api/admin/usuarios	UsuarioGestion o 409 DUPLICADO, DOCENTE_CON_CUENTA, ULTIMO_ADMIN o USUARIO_ACTUAL	RF-11
GET /api/mi/perfil y GET /api/mi/horario	Perfil del docente y sus sesiones del horario publicado o 404 SIN_PUBLICAR	RF-12, RF-14
GET y PUT /api/mi/disponibilidad	Franjas válidas y franjas marcadas o 400 RD-03	RF-14
PUT /api/mi/contrasena	204 o 400 CONTRASENA_INCORRECTA	RF-14

Nota. Las rutas /api/mi exigen el rol DOCENTE. Las demás salvo el inicio de sesión y GET /api/parametros exigen el rol ADMIN. Sin token la respuesta es 401 con el código NO_AUTENTICADO y con el rol equivocado es 403.

Diccionario de Datos

El esquema lo crea Flyway al arrancar el backend con tres scripts. `V1__esquema.sql` crea las 19 tablas, `V2__datos_base.sql` carga los catálogos de características y de restricciones y los valores por defecto de los parámetros y `V3__usuarios_y_publicacion.sql` agrega los perfiles de usuario y la publicación del horario. El mismo SQL corre en PostgreSQL 16 y en H2 en modo PostgreSQL. Las tablas se agrupan en cuatro bloques. El bloque de inventarios guarda lo que llega en los CSV. El bloque de configuración guarda los parámetros y las franjas vigentes. El bloque de ejecuciones guarda cada horario generado con sus conflictos. El bloque de seguridad guarda las cuentas de los dos perfiles. El diagrama entidad relación está en la Figura D1 del Documento 2. Anexos Técnicos (PN2_Grupo5_Doc2_Anexos.pdf).

Inventarios

docente

Un registro por docente del archivo docentes.csv.

Tabla 4
Columnas de la tabla docente

Columna	Tipo	Clave	Descripción y restricciones
id_docente	VARCHAR(10)	PK	Identificador del docente, por ejemplo D01
nombre	VARCHAR(80)		Obligatorio
dedicacion	VARCHAR(20)		Dato informativo del archivo
escalafon	VARCHAR(30)		Dato informativo del archivo

disponibilidad_docente

Franjas en que el docente puede dictar. Se guardan por día y hora de inicio y no por índice de franja para que sobrevivan a un cambio de la grilla.

Tabla 5
Columnas de la tabla disponibilidad_docente

Columna	Tipo	Clave	Descripción y restricciones
id_docente	VARCHAR(10)	PK, FK	Referencia a docente
dia	VARCHAR(10)	PK	Día de la semana
hora_inicio	TIME	PK	Hora de inicio del bloque

aula

Un registro por aula del archivo aulas.csv.

Tabla 6
Columnas de la tabla aula

Columna	Tipo	Clave	Descripción y restricciones
id_aula	VARCHAR(10)	PK	Identificador del aula
capacidad	SMALLINT		CHECK capacidad mayor que cero

caracteristica

Catálogo de características de aula. Se carga con V2 y hoy tiene COMPUTADORES y SILLAS_MOVILES.

Tabla 7
Columnas de la tabla caracteristica

Columna	Tipo	Clave	Descripción y restricciones
id_caracteristica	VARCHAR(20)	PK	Código de la característica
descripcion	VARCHAR(60)		Texto para la interfaz

aula_caracteristica

Características que tiene cada aula. Resuelve la relación de muchos a muchos.

Tabla 8
Columnas de la tabla aula_caracteristica

Columna	Tipo	Clave	Descripción y restricciones
id_aula	VARCHAR(10)	PK, FK	Referencia a aula
id_caracteristica	VARCHAR(20)	PK, FK	Referencia a caracteristica

curso

Un registro por asignatura del archivo cursos.csv.

Tabla 9
Columnas de la tabla curso

Columna	Tipo	Clave	Descripción y restricciones
id_curso	VARCHAR(10)	PK	Identificador del curso
nombre	VARCHAR(80)		Obligatorio
num_sesiones_semanales	SMALLINT		CHECK entre 1 y 4 (RD-07)

curso_requisito_aula

Características que el curso exige del aula (RD-05).

Tabla 10
Columnas de la tabla curso_requisito_aula

Columna	Tipo	Clave	Descripción y restricciones
id_curso	VARCHAR(10)	PK, FK	Referencia a curso
id_caracteristica	VARCHAR(20)	PK, FK	Referencia a caracteristica

habilitacion_docente

Docentes habilitados para dictar cada curso (S-06). Si un curso no tiene filas aquí cualquier docente puede dictarlo.

Tabla 11
Columnas de la tabla habilitacion_docente

Columna	Tipo	Clave	Descripción y restricciones
id_curso	VARCHAR(10)	PK, FK	Referencia a curso
id_docente	VARCHAR(10)	PK, FK	Referencia a docente

grupo

Un registro por grupo del archivo grupos.csv.

Tabla 12
Columnas de la tabla grupo

Columna	Tipo	Clave	Descripción y restricciones
id_grupo	VARCHAR(15)	PK	Identificador del grupo, por ejemplo MAT1-G1
id_curso	VARCHAR(10)	FK	Referencia a curso
matriculados	SMALLINT		CHECK mayor o igual a cero

sesion

Sesiones semanales de cada grupo. Las crea el cargador según num_sesiones_semanales del curso.

Tabla 13
Columnas de la tabla sesion

Columna	Tipo	Clave	Descripción y restricciones
id_sesion	VARCHAR(20)	PK	Identificador de la sesión
id_grupo	VARCHAR(15)	FK	Referencia a grupo
numero_sesion	SMALLINT	UNIQUE con id_grupo	Número de la sesión dentro del grupo

Configuración

parametro_configuracion

Parámetros del semestre (S-04). Se editan desde la pantalla de parámetros.

Tabla 14
Columnas de la tabla parametro_configuracion

Columna	Tipo	Clave	Descripción y restricciones
clave	VARCHAR(30)	PK	CAP_MAX, TOL, MIN_CIERRE, MAX_SESIONES, DUR_SESION, APERTURA_SEMANA, CIERRE_SEMANA, CIERRE_SABADO, INICIO_BLOQUEO o FIN_BLOQUEO
valor	VARCHAR(30)		Valor como texto
tipo_dato	VARCHAR(10)		CHECK ENTERO, DECIMAL u HORA

Nota. Valores por defecto 40, 0.10, 10, 4, 2, 07:00, 22:00, 13:00, 12:00 y 13:00 en el orden de las claves.

franja_horaria

Franjas válidas vigentes. RepositorioParametros la reescribe cada vez que cambian los parámetros. Con los valores por defecto tiene 32 filas.

Tabla 15
Columnas de la tabla franja_horaria

Columna	Tipo	Clave	Descripción y restricciones
id_franja	SMALLINT	PK	Índice de la franja desde 0
dia	VARCHAR(10)	UNIQUE con hora_inicio	CHECK LUNES a SABADO
hora_inicio	TIME		Inicio del bloque
hora_fin	TIME		Fin del bloque

restriccion

Catálogo de códigos de regla. Lo usan los conflictos y los hallazgos para mostrar la descripción de cada código.

Tabla 16
Columnas de la tabla restriccion

Columna	Tipo	Clave	Descripción y restricciones
codigo	VARCHAR(6)	PK	RD-01 a RD-08, RD-10, RV-01 y RV-02
tipo	VARCHAR(10)		CHECK DURA o VALIDACION
descripcion	VARCHAR(160)		Texto que ve el administrador

Ejecuciones

ejecucion_horario

Una fila por cada vez que se genera un horario. Permite saber quién generó cada horario y cuándo (RNF-06).

Tabla 17
Columnas de la tabla ejecucion_horario

Columna	Tipo	Clave	Descripción y restricciones
id_ejecucion	INTEGER	PK	Identidad generada por la base
id_usuario	INTEGER	FK	Referencia a usuario
fecha_hora	TIMESTAMP		Momento de la generación
estado	VARCHAR(10)		CHECK COMPLETO o PARCIAL
nodos_explorados	BIGINT		Nodos del backtracking
duracion_ms	INTEGER		Duración total en milisegundos
publicado	BOOLEAN		Verdadero en la única ejecución publicada a los docentes. Por defecto falso (V3)

asignacion

Una fila por sesión ubicada en una ejecución.

Tabla 18
Columnas de la tabla asignacion

Columna	Tipo	Clave	Descripción y restricciones
id_ejecucion	INTEGER	PK, FK	Referencia a ejecucion_horario
id_sesion	VARCHAR(20)	PK, FK	Referencia a sesion
id_docente	VARCHAR(10)	FK	Referencia a docente
id_aula	VARCHAR(10)	FK	Referencia a aula

Columna	Tipo	Clave	Descripción y restricciones
dia	VARCHAR(10)		Día asignado
hora_inicio	TIME		Inicio asignado
hora_fin	TIME		Fin asignado

Nota. UNIQUE (id_ejecucion, id_docente, dia, hora_inicio) protege RD-01. UNIQUE (id_ejecucion, id_aula, dia, hora_inicio) protege RD-02.

conflicto

Una fila por sesión que quedó sin ubicar en una ejecución.

Tabla 19

Columnas de la tabla conflicto

Columna	Tipo	Clave	Descripción y restricciones
id_conflicto	INTEGER	PK	Identidad generada por la base
id_ejecucion	INTEGER	FK, UNIQUE con id_sesion	Referencia a ejecucion_horario
id_sesion	VARCHAR(20)	FK	Referencia a sesion

conflicto_restriccion

Causas de cada conflicto con el número de tuplas que bloqueó cada código.

Tabla 20

Columnas de la tabla conflicto_restriccion

Columna	Tipo	Clave	Descripción y restricciones
id_conflicto	INTEGER	PK, FK	Referencia a conflicto
codigo	VARCHAR(6)	PK, FK	Referencia a restriccion
tuplas_bloqueadas	INTEGER		CHECK mayor que cero

hallazgo_validacion

Hallazgos de RV-01 y RV-02 encontrados antes de generar.

Tabla 21

Columnas de la tabla hallazgo_validacion

Columna	Tipo	Clave	Descripción y restricciones
id_ejecucion	INTEGER	PK, FK	Referencia a ejecucion_horario

Columna	Tipo	Clave	Descripción y restricciones
id_grupo	VARCHAR(15)	PK, FK	Referencia a grupo
codigo	VARCHAR(6)	PK, FK	Referencia a restriccion
mensaje	VARCHAR(200)		Texto del hallazgo

Seguridad

usuario

Usuarios que pueden entrar al sistema con perfil ADMIN o DOCENTE. Cada cuenta DOCENTE queda ligada a un docente.

Tabla 22

Columnas de la tabla usuario

Columna	Tipo	Clave	Descripción y restricciones
id_usuario	INTEGER	PK	Identidad generada por la base
nombre_usuario	VARCHAR(40)	UNIQUE	Nombre para iniciar sesión
hash_contrasena	VARCHAR(100)		Hash bcrypt, nunca la clave en claro
rol	VARCHAR(15)		CHECK ADMIN o DOCENTE (V3)
id_docente	VARCHAR(10)	FK	Referencia a docente. CHECK obligatorio si el rol es DOCENTE (V3)
activo	BOOLEAN		Una cuenta inactiva no puede iniciar sesión. Por defecto verdadero (V3)

Archivos de Intercambio

El intercambio de archivos es solo en CSV. Los archivos usan coma como separador, codificación UTF-8 y una primera fila de encabezados. Las listas dentro de un campo van separadas por punto y coma. La Tabla 23 muestra el encabezado exacto que exige el cargador y un ejemplo de fila.

Tabla 23

Formato de los archivos CSV de entrada y salida

Archivo	Encabezado	Ejemplo de fila
docentes.csv	id_docente,nombre,dedicacion,escalafon,disponibilidad	D01,Dr. Gómez,MEDIO,ASOCIADO,MAR 07:00;MAR 09:00;MIE 09:00

Archivo	Encabezado	Ejemplo de fila
aulas.csv	id_aula,capacidad,caracteristicas	A301,45,COMPUTADORES
cursos.csv	id_curso,nombre,num_sesiones,requisitos,docentes_habilitados	CAL2,Cálculo II,2,,D01
grupos.csv	id_grupo,id_curso,matriculados	CAL2-G2,CAL2,35
horario.csv (salida)	curso,grupo,sesion,docente,aula,dia,hora_inicio,hora_fin	Cálculo II,CAL2-G2,1,Dr. Gómez,A204,MIERCOLES,09:00,11:00

Nota. Los días válidos son LUN, MAR, MIE, JUE, VIE y SAB. En disponibilidad la palabra TODAS equivale a las 32 franjas. Un campo docentes_habilitados vacío significa que cualquier docente puede dictar el curso (S-06).

Cada fila que no cumple el formato se rechaza con el número de fila y un código. El código FORMATO indica un dato vacío, repetido o mal escrito. REFERENCIA indica un identificador que no existe en otro archivo. RD-03 indica una franja que no pertenece a la grilla. RD-04 indica un docente sin ninguna franja disponible. RD-07 indica un curso con un número de sesiones fuera del rango permitido. Las filas válidas se cargan aunque otras se rechacen.

Guía de Instalación desde Cero

Esta guía parte de un equipo que no tiene nada del proyecto. La forma recomendada es con Docker porque levanta los tres servicios con un solo comando. Al final hay una ruta alternativa sin Docker para el caso en que el equipo no pueda instalarlo.

Instalación con Docker

Paso 1. Instalar Docker. En Windows y macOS se instala Docker Desktop desde la página oficial y se deja abierto. En Linux se instalan Docker Engine y el complemento Compose v2. Para comprobar la instalación se abre una terminal y se ejecutan los dos comandos siguientes. Ambos deben responder con un número de versión.

```
docker --version
docker compose version
```

Paso 2. Instalar Git y clonar el repositorio. Se instala Git y se descarga el proyecto en una carpeta de trabajo.

```
git clone https://github.com/jesusBeltran79/ProgHorarioClase.git
cd ProgHorarioClase
```

Paso 3. Revisar los puertos. Los puertos 4200, 8080 y 5432 deben estar libres. Si el equipo ya tiene un PostgreSQL local en el 5432 hay que detenerlo o cambiar el puerto publicado en `docker-compose.yml`.

Paso 4. Construir y arrancar. Desde la raíz del repositorio se ejecuta el comando siguiente. La primera vez tarda varios minutos porque descarga las imágenes base, compila el backend con Maven y compila la interfaz con Node.js. Las siguientes veces es mucho más rápido.

```
docker compose up --build
```

Paso 5. Comprobar que todo está arriba. Cuando la terminal deje de mostrar mensajes de arranque se abre en el navegador la dirección de salud del backend. Debe responder con el estado UP. Después se abre la interfaz web.

```
http://localhost:8080/actuator/health   ->  {"status":"UP"}
http://localhost:4200                   ->  pantalla de inicio de sesión
```

Paso 6. Iniciar sesión. El usuario inicial es `admin` con la clave `Horarios2026*`. Antes de usar el sistema fuera de una demostración se debe cambiar esa clave y el secreto de los tokens con las variables de entorno de la Tabla 24.

Paso 7. Cargar los datos de demostración. En la carpeta `datos/demo` están los cuatro archivos del conjunto de sustentación con 10 docentes, 5 aulas y 20 asignaturas. El Documento 4. Manual de Usuario (PN2_Grupo5_Doc4_ManualUsuario.pdf) explica cómo cargarlos y generar el primer horario.

El archivo `docker-compose.yml` define tres servicios. El servicio `db` usa la imagen postgres:16-alpine y guarda los datos en el volumen `datos-postgres`. El servicio `backend` se construye desde backend/Dockerfile en dos etapas, compila con Maven y corre el jar con `java -Xss64m`. El servicio `frontend` se construye desde frontend/Dockerfile, compila Angular con Node.js 20 y lo sirve con nginx, que redirige las rutas /api y /actuator al backend. Cada servicio espera a que el anterior esté sano antes de arrancar. La Figura E2 del Documento 2. Anexos Técnicos (PN2_Grupo5_Doc2_Anexos.pdf) muestra el despliegue y

docs/evidencias/sistema/contenedores.txt muestra los tres contenedores arriba y sanos en la ejecución de prueba del equipo.

Instalación sin Docker

Esta ruta usa el perfil `h2` del backend, que reemplaza PostgreSQL por una base en memoria compatible. Los datos se pierden al detener el backend. Sirve como respaldo para una demostración o para desarrollo.

Paso 1. Instalar Java 21 (Eclipse Temurin), Maven 3.9 y Node.js 20 LTS y comprobar las versiones con `java -version`, `mvn -v` y `node -v`.

Paso 2. Compilar y arrancar el backend con el perfil de respaldo.

```
cd backend
mvn spring-boot:run -Dspring-boot.run.profiles=h2
```

Paso 3. En otra terminal instalar las dependencias de la interfaz y arrancarla.

```
cd frontend
npm install
npm start
```

El servidor de desarrollo de Angular redirige /api y /actuator al backend en el puerto 8080 con el archivo `frontend/proxy.conf.json`. La interfaz queda en http://localhost:4200. Para usar un PostgreSQL local en lugar de H2 se arranca el backend sin el perfil `h2` y se definen `DB_URL`, `DB_USUARIO` y `DB_CLAVE`.

Paso 4. Para correr solo las pruebas automáticas del backend no hace falta la interfaz. Desde la carpeta `backend` se ejecuta `mvn verify`. Las pruebas usan H2 y no necesitan PostgreSQL.

Operación

Variables de Entorno

El backend lee su configuración de `application.yml` y permite reemplazar cada valor con una variable de entorno. La Tabla 24 lista las que importan para operar el sistema.

Tabla 24

Variables de entorno del backend

Variable	Valor por defecto	Uso
DB_URL	jdbc:postgresql://localhost:5432/horarios	Dirección de la base
DB_USUARIO y DB_CLAVE	horarios y horarios	Credenciales de la base
PUERTO	8080	Puerto del backend
JWT_SECRETO	Secreto de desarrollo del grupo	Firma de los tokens. Debe tener al menos 32 caracteres y cambiarse fuera de la demostración
JWT_MINUTOS	480	Duración del token
ADMIN_USUARIO y ADMIN_CLAVE	admin y Horarios2026*	Usuario administrador que se crea al primer arranque
MOTOR_LIMITE_MS	60000	Tiempo máximo de una generación
CORS_ORIGENES	http://localhost:4200	Orígenes que pueden llamar a la API

Comportamientos que Conviene Conocer

Una carga nueva de inventarios reemplaza los anteriores y borra el historial de ejecuciones, porque las asignaciones guardadas apuntan a sesiones que dejan de existir. Los rechazos de la última carga se guardan en memoria y no en la base de datos, de modo que se pierden al reiniciar el backend. Las reglas RV-01 y RV-02 informan los grupos con problemas pero no los excluyen de la generación. Si los parámetros cambian después de generar, los horarios anteriores ya no se pueden consultar ni reubicar y el sistema responde GRILLA_CAMBIADA.

Una carga nueva por CSV conserva las cuentas de los docentes que siguen en el archivo y borra la cuenta de los que ya no están. Cualquier cambio de catálogo desde la interfaz también borra los horarios generados. El cambio de disponibilidad que hace un docente desde su portal no borra nada y se aplica en la siguiente generación. Un token emitido sigue vigente hasta vencer aunque la cuenta se desactive después.

Tareas Frecuentes

Para detener el sistema se presiona Ctrl+C en la terminal donde corre o se ejecuta `docker compose down` en otra. Los datos de PostgreSQL se conservan entre arranques. Para empezar con una base vacía se ejecuta `docker compose down -v`, que borra también el volumen de datos.

Los registros del backend se ven con `docker compose logs -f backend`. Cada generación deja en la tabla `ejecucion_horario` el usuario, la fecha, el estado, los nodos explorados y la duración. Eso permite revisar el historial sin abrir los registros.

Las mediciones de estrés se repiten fuera de la aplicación con el programa de consola `PruebaEstres`. La integración continua lo ejecuta en cada envío al repositorio después de las pruebas JUnit y publica los resultados como artefacto.

```
cd backend
mvn -q package -DskipTests
java -Xss64m -cp target/classes co.edu.unbosque.horarios.app.PruebaEstres
target/evidencias/estres 60000
```

La prueba de estrés genera cinco conjuntos por escala con semillas fijas en dos perfiles de datos y escribe docs/evidencias/estres/estres.csv con una fila por corrida y resumen.txt con el promedio, la mediana y el peor caso. Los resultados se analizan en el escenario de escalabilidad del Documento 1. Informe de Ingeniería (PN2_Grupo5_Doc1_Informe.pdf).

El parámetro `-Xss64m` amplía la pila de la máquina virtual porque el backtracking es recursivo y en 10x la profundidad llega a cientos de sesiones.

Guía de Mantenimiento del Algoritmo de Restricciones

Esta sección explica cómo cambiar las reglas del motor sin romperlo. El diseño se pensó para que los cambios frecuentes no toquen `MotorAsignacion`. Cada restricción es una estrategia del patrón Strategy (Gamma et al., 1994) y el motor solo conoce su interfaz. La Tabla 25 dice dónde se hace cada tipo de cambio.

Tabla 25
Dónde se hace cada tipo de cambio

Cambio	Dónde se hace	Recompilar
Tope de matriculados, tolerancia o mínimo de cierre	Pantalla de parámetros o PUT /api/parametros	No
Horario de apertura, cierre, bloqueo del mediodía o duración de la sesión	Pantalla de parámetros o PUT /api/parametros	No
Nueva restricción dura	Clase nueva en restricciones, una línea en ConfiguracionMotor y un script de Flyway	Sí
Nueva regla de validación de entrada	Clase nueva en validacion y una línea en ConfiguracionMotor	Sí
Nueva característica de aula	Valor en el enum CaracteristicaAula y fila en el catálogo caracteristica	Sí

Cómo Agregar una Restricción Dura

El ejemplo es RD-09, que prohíbe que dos sesiones del mismo grupo queden el mismo día. Es la regla que el equipo usó para demostrar la modificabilidad. Los pasos son cinco y ninguno toca el motor.

Paso 1. Decidir si la regla es local. Una regla es local si para decidir le basta la tupla candidata de docente, aula y franja. RD-04, RD-05 y RD-06 son locales. Una regla no es local si necesita mirar el horario ya construido, como los choques. RD-09 no es local porque debe revisar las otras sesiones del grupo. La decisión importa porque el motor usa las reglas locales en el conteo inicial y las evalúa primero en el backtracking. Si una regla que mira el horario se declara local, el motor la llama en el conteo inicial con el estado en nulo y falla.

Paso 2. Crear la clase. La clase va en el paquete `restricciones`, implementa `RestriccionDura`, es final y no guarda estado propio. El nombre sigue la convención Restriccion más el concepto más el código.

```
public final class RestriccionMismoDiaRD09 implements RestriccionDura {
    @Override public String getCodigo() { return "RD-09"; }
    @Override public boolean esLocal() { return false; }
    @Override
    public boolean esValida(Asignacion c, EstadoHorario e, ConfiguracionParametros cfg)
{
        for (Asignacion a : e.getAsignaciones())
            if (a.getSesion().getGrupo() == c.getSesion().getGrupo()
```

```
                    && a.getFranja().getDia() == c.getFranja().getDia()) return false;
        return true;
    }
}
```

Paso 3. Registrar la regla. Se agrega una línea con `new RestriccionMismoDiaRD09()` en la lista de `ConfiguracionMotor.catalogoRestricciones`. El orden de la lista no altera el resultado porque el motor pone primero las reglas locales al construirse.

Paso 4. Registrar el código en la base. Las tablas `conflicto_restriccion` y `hallazgo_validacion` tienen una clave foránea hacia `restriccion`. Si el código nuevo no está en ese catálogo, la primera ejecución con un conflicto causado por la regla falla al guardarse. Por eso se crea un script nuevo de Flyway. Nunca se edita V1 ni V2 porque Flyway rechaza un script ya aplicado que cambió. El diff de evidencia docs/evidencias/modificabilidad/rd09.diff no trae este script porque en esa corrida RD-09 no causó ningún conflicto que hubiera que guardar. Para dejar la regla en producción el script es obligatorio.

```
-- backend/src/main/resources/db/migration/V3__restriccion_rd09.sql
INSERT INTO restriccion (codigo, tipo, descripcion) VALUES
    ('RD-09', 'DURA', 'Dos sesiones del mismo grupo no pueden quedar el mismo día');
```

Paso 5. Probar. Se agrega un caso en `RestriccionesDurasTest` con dos sesiones del mismo grupo el mismo día y se comprueba que la regla responde falso. Luego se corre `mvn verify` completo para confirmar que las demás reglas y el motor siguen en verde. La diferencia de versiones de este cambio debe mostrar cero líneas modificadas en `MotorAsignacion`. La evidencia de ese diff está en el escenario de modificabilidad del Documento 1. Informe de Ingeniería (PN2_Grupo5_Doc1_Informe.pdf).

Cómo Agregar una Regla de Validación

Las reglas RV no forman parte del backtracking. Se evalúan una vez por grupo antes de generar y sus hallazgos no detienen el proceso. Para agregar una se crea una clase en el paquete `validacion` que implemente `ReglaValidacionEntrada`, se registra en el bean

`validadorEntrada` de `ConfiguracionMotor` y se agrega su código al catálogo `restriccion` con tipo VALIDACION mediante un script de Flyway nuevo.

Qué no Conviene Tocar

Hay tres puntos sensibles. El primero es el ciclo de `MotorAsignacion.backtrack`. Cambiarlo afecta a todas las reglas a la vez y obliga a repetir las pruebas de estrés. El segundo es `EstadoHorario`. Si se agrega una estructura de ocupación nueva hay que mantenerla también en `removerUltimaAsignacion` o el retroceso deja datos sucios. El tercero es el cálculo de `calcularUmbralMaximoGrupo`. Usa el techo de CAP_MAX por uno más TOL con un pequeño margen para que 40 por 1,10 no termine en 45 por el error del punto flotante. Con tope 40 da 44 y con tope 35 da 39.

Si se quiere mejorar el desempeño del conteo inicial, el camino que quedó identificado es precalcular por franja cuántos docentes y cuántas aulas sirven. Eso exige que cada regla declare de qué elemento de la tupla depende y por tanto cambia el contrato de `RestriccionDura`. Es un cambio mayor que debe medirse otra vez en 1x, 3x, 5x y 10x antes de adoptarse.

Solución de Problemas

Tabla 26

Problemas frecuentes y cómo resolverlos

Síntoma	Causa probable	Solución
docker compose falla con un puerto en uso	Otro programa usa el 4200, el 8080 o el 5432	Detener ese programa o cambiar el puerto publicado en docker-compose.yml
El backend no arranca y menciona Flyway	Un script de migración ya aplicado fue modificado	Restaurar el script original y crear uno nuevo con la versión siguiente
La API responde 401 NO_AUTENTICADO	Falta el token o venció	Iniciar sesión de nuevo
La API responde 403 SIN_PERMISO	La cuenta tiene un perfil que no puede usar esa ruta, por ejemplo un DOCENTE que intenta generar	Entrar con una cuenta ADMIN
El portal del docente responde 404 SIN_PUBLICAR	No hay ningún horario publicado	El ADMIN publica un horario COMPLETO

Síntoma	Causa probable	Solución
Un docente no puede iniciar sesión con su cuenta	La cuenta está inactiva o su docente fue borrado o no vino en la última carga CSV	El ADMIN revisa la cuenta en Usuarios y la activa o la crea de nuevo
Generar responde 409 SIN_DATOS	No se han cargado inventarios	Cargar los cuatro archivos CSV
Exportar responde 409 HORARIO_PARCIAL	Quedan sesiones sin ubicar y no se cumple RD-08	Resolver los conflictos con la reubicación asistida y exportar de nuevo
Consultar un horario responde 409 GRILLA_CAMBIADA	Los parámetros cambiaron después de generar ese horario	Generar un horario nuevo con la grilla vigente
El motor termina con StackOverflowError en escalas grandes	La pila por defecto de la JVM es corta para la recursión	Ejecutar con -Xss64m

Referencias

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.

Russell, S. y Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.